Rapport Utilisateur & Technique - SGEB (Système de Gestion des Emprunts de Bibliothèque)

1. Introduction

Ce document présente le **Système de Gestion des Emprunts de Bibliothèque (SGEB)**. L'objectif du logiciel est de permettre à un administrateur de bibliothèque de gérer :

- Les documents (livres, magazines),
- Les adhérents,
- Les emprunts et retours,
- Les pénalités liées aux retards.

Le projet a été développé en **Java** (version 17) en respectant les principes de la Programmation Orientée Objet (POO). L'interface graphique a été réalisée avec **JavaFX**. La persistance des données est assurée par une base de données relationnelle **SQLite**.

2. Installation et Lancement

Prérequis

- Un ordinateur disposant de Java (JRE ou JDK) version 17 ou supérieure.
- Le fichier archive fourni Tsitsiev.zip.

Procédure de lancement

L'application est fournie sous forme de fichier **JAR exécutable**, ne nécessitant pas d'environnement de développement (comme Eclipse) pour fonctionner.

1. Extraire l'archive .zip.
2. Ouvrir un terminal (ou invite de commande) dans le dossier extrait.
3. Lancer la commande suivante : java -jar Tsitsiev.jar
4. Au premier lancement, le fichier de base de données bibliotheque.db sera créé automatiquement à la racine du dossier.

3. Guide de l'Utilisateur

L'application s'ouvre sur un tableau de bord principal divisé en trois zones fonctionnelles : **Documents**, **Adhérents** et **Circulation**.

C’est structuré de la manière suivante :

Documents :

- Voir les documents
- Ajouter un document
- Modifier un document
- Supprimer un document
- Rechercher par titre / auteur / genre

Adhérents :

- Voir les adhérents
- Ajouter un adhérent
- Modifier un adhérent
- Supprimer un adhérent
- Voir l’historique d’un adhérent
- Voir / modifier le statut (pénalité)
- Voir les retards

Circulation :

- Emprunter un document
- Retourner un document
- Voir les emprunts en cours
- Quitter l’application

Chaque bouton ouvre une nouvelle fenêtre dédiée à l’action choisie.

4. Gestion des documents

4.1 Voir les documents

Le bouton «Voir les documents » affiche une table contenant tous les documents présents dans la base.

Les colonnes affichées sont :

- Type (Livre / Magazine),
- Titre,
- Auteur,
- Genre,
- Détails (ISBN et nombre de pages pour un livre, numéro et périodicité pour un magazine),
- Disponibilité (disponible / emprunté).

4.2 Ajouter un document

Le bouton « Ajouter un document » ouvre une fenêtre permettant de :

1. Choisir le type : Livre ou Magazine.
2. Saisir les champs génériques :
    - Titre (obligatoire),
    - Auteur (obligatoire),
    - Genre (optionnel).
3. Saisir les champs spécifiques :
    - Livre : ISBN (optionnel), nombre de pages (optionnel),
    - Magazine : numéro (optionnel), périodicité en jours (optionnel).

Après validation, le document est :

- enregistré dans la base de données,
- ajouté en mémoire,
- affiché dans la liste des documents.

En cas d'erreur (titre/auteur manquant, format invalide, doublon), un message explicite est affiché. Un document est unique par le couple (Titre, Auteur). En effet, plusieurs documents peuvent avoir le meme titre, mais l'auteur doit être différent. Cela me semblait la meilleur solution (en termes d'utilisation et de logique).

4.3 Modifier un document

Le bouton « Modifier un document » ouvre d'abord une fenêtre de recherche par titre et auteur, puis une fenêtre d'édition.

On peut modifier :

- le titre,
- l'auteur,
- le genre,
- la disponibilité,
- l'ISBN et le nombre de pages (livre),
- le numéro et la périodicité (magazine).

Les modifications sont enregistrées en base via BibliothequeManager et DatabaseManager.
En cas de doublon (autre document avec même titre + auteur), une alerte est affichée.

NB : Concernant l'ISBN et le nombre de pages (pour les livres) ainsi que le numéro et la périodicité (pour les magazines), j'ai choisi de les laisser optionnels à la création pour offrir plus de souplesse si l'utilisateur n'a pas toutes les infos tout de suite. En revanche, une fois qu'une valeur est entrée, elle ne peut plus être remise à vide. J'ai mis en place cette logique car cela me semblait le plus ergonomique. En effet, une autre solution technique aurait obligé l'utilisateur à taper manuellement le chiffre "0" pour effacer une donnée, ce qui n'est pas du tout naturel ni pratique à l'usage. De plus, ce fonctionnement permet de sécuriser la saisie en empêchant qu'une information importante (dont ISBN par exemple), déjà enregistrée, ne soit supprimée par erreur lors d'une modification.

4.4 Supprimer un document

Le bouton « Supprimer un document » permet de supprimer un document uniquement s'il n'est pas emprunté (donc avec le statut disponible).

La suppression :

- efface le document de la base (DELETE),
- le retire de la liste en mémoire.

4.4 Chercher un document par Titre, Auteur ou Genre :

Le bouton « Cherche un document » permet de vérifier si un document est présent dans la bibliothèque, et s'il est disponible ou non. Donc, cela affiche une table contenant tous les documents présents dans la base, avec :

- le titre,
- l'auteur,
- le genre,
- la disponibilité,

NB : Concernant la recherche par titre, j'ai ajouté une fonctionnalité pour gérer les erreurs ou les imprécisions. Si le titre exact n'est pas trouvé, le système affiche automatiquement tous les documents qui *contiennent* le mot saisi. De plus, ni les espaces ni les majuscules n'ont de l'importances.

5. Gestion des adhérents

5.1 Voir les adhérents

Le bouton «Voir les adhérents » affiche une table avec :

- Nom,
- prénom,
- Email (unique),

5.2 Ajouter un adhérent

Le bouton « Ajouter un adhérent » ouvre une fenêtre de saisie :

- Nom,
- Prénom,
- Email (unique)

BibliothequeManager.ajouterAdherent() vérifie qu'aucun autre adhérent n'a déjà le même email avant d'insérer en base.

5.3 Modifier un adhérent

Le bouton « Modifier un adhérent » permet :

- de chercher un adhérent par email,
- de modifier son nom, prénom et email (sauf si le mail existe déjà),
- de gérer son statut de pénalité :
    - En règle → possibilité d'appliquer une pénalité,
    - Pénalisé → possibilité de lever la sanction.

Les modifications sont propagées à la base (avec updateAdherent).

NB : Certains noms de méthodes dans la classe de gestion de la BD sont en anglais, afin d'améliorer la lisibilité et de faciliter la compréhension. Le reste du code est rédigé en français.

5.4 Supprimer un adhérent

Le bouton « Supprimer un adhérent » :

1. Demande l'email de l'adhérent à supprimer.
2. Vérifie qu'il existe.
3. Vérifie qu'il n'a pas d'emprunt en cours (retour non enregistré).
4. Demande une confirmation finale avant la suppression.

5.5 Historique d'un adhérent

Le bouton «Historique d'un adhérent» demande d'entrer le email d'un adhérent, ensuite s'affiche une table avec les emprunts qu'il a pu faire. Les informations suivantes y figurent :

- Le titre du document,
- La date d'emprunt,
- La date prévue du retour,
- Et la date réelle du retour.

5.5 Voir le statut d'un adhérent

Le bouton «Voir le statut d'un adhérent» demande d'entrer le email d'un adhérent, ensuite un message s'affiche pour indiquer si l'adhèrent à une pénalité ou non.

5.5 Voir les retards

Le bouton «Voir les retards » affiche une table avec tous les emprunts en retards en cours. Les informations suivantes y figurent :

- Le titre du document,
- Nom et Prénom de l'adhérent,
- La date prévue du retour,
- Nombre de jours de retard.

6. Circulation

6.1 Emprunter un document

Via « Emprunter un document », l'utilisateur saisit :

- l'email de l'adhérent,
- le titre et l'auteur du document qu'il veut emprunter.

Le système :

1. Vérifie que l'adhérent existe.
2. Vérifie que le document existe et est disponible.
3. Vérifie que l'adhérent n'a pas de pénalité.
4. Vérifie qu'il n'a pas déjà 5 emprunts en cours.

Et BibliothequeManager

5. Crée un emprunt avec :
    - la date d'emprunt (date du jour),
    - la date de retour prévue (3 semaines plus tard).

Un message informe l'utilisateur du succès ou de la raison du refus.

6.2 Voir les emprunts en cours

Le bouton « Voir les emprunts en cours » affiche tous les emprunts dont la date de retour réelle est nulle (non rendus).

6.3 Retourner un document

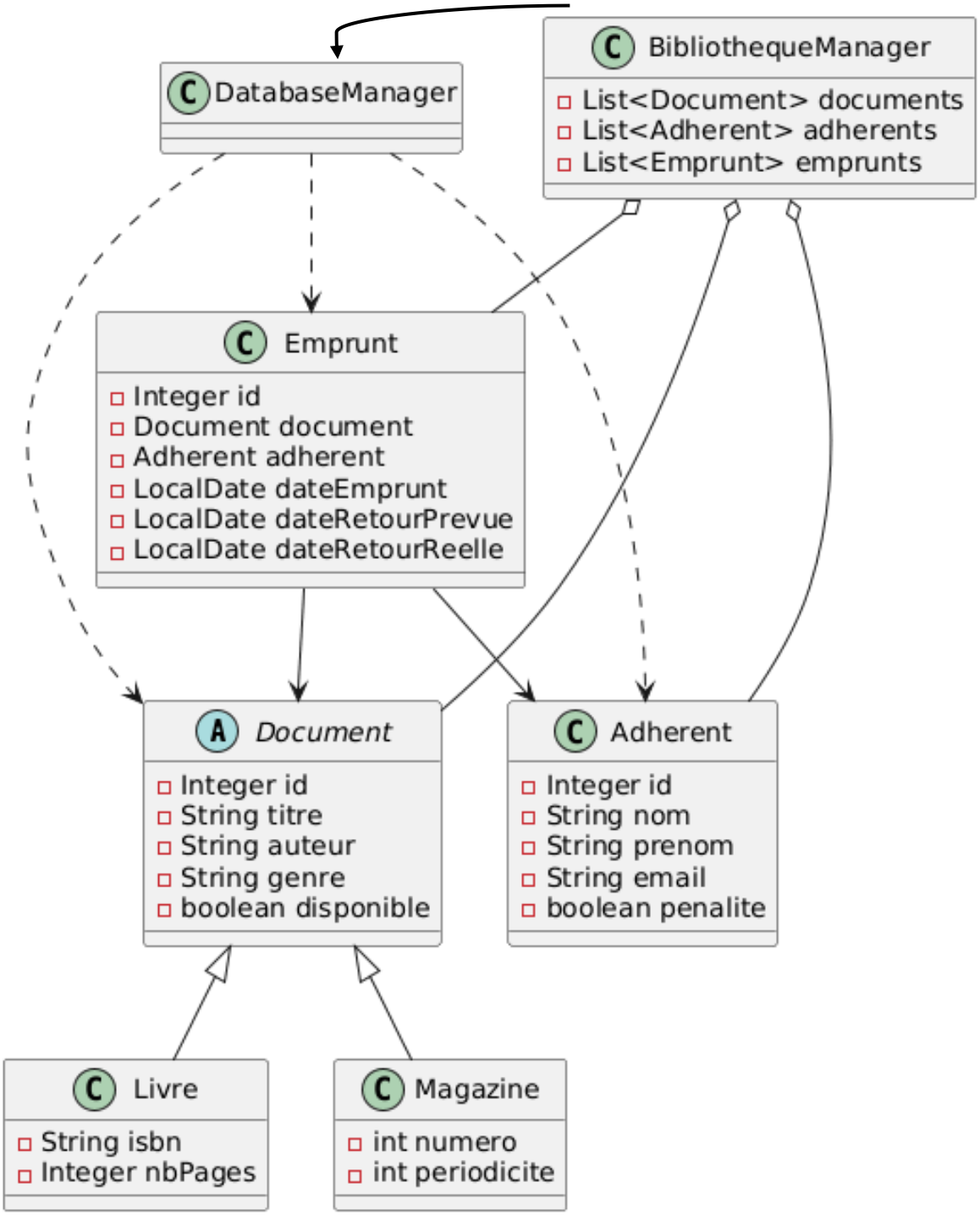
Le bouton « Retourner un document » ouvre une fenêtre avec :

1. Saisie de l'email de l'adhérent et du document (titre + auteur).
2. Bouton « Vérifier le retard » :
    - calcule le nombre de jours de retard,
    - affiche le montant de l'amende (0,50 €/jour),
    - active une case « Amende réglée immédiatement ? ».
3. Bouton « Valider le retour » :
    - appelle la logique de retour,
    - met à jour :
        - la date de retour réelle,
        - le statut du document (disponible),
        - le statut de pénalité (true s'il ne règle pas immédiatement).

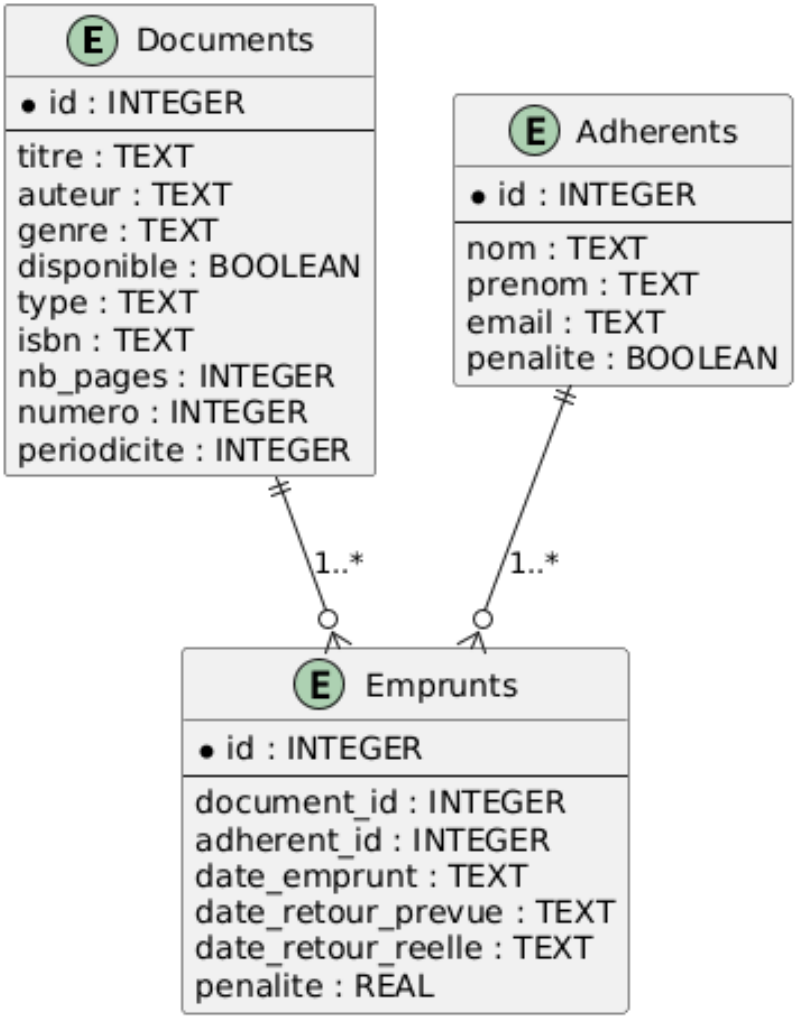
NB : Mais il pourra régler un autre jour. En effet, il suffit de modifier l'adhérent, et de lever la sanction une fois qu'il a payé. Donc, un adhérent pénalisé ne peut plus emprunter tant que sa pénalité n'est pas levée.

7. Conception Technique (Diagrammes ULM)

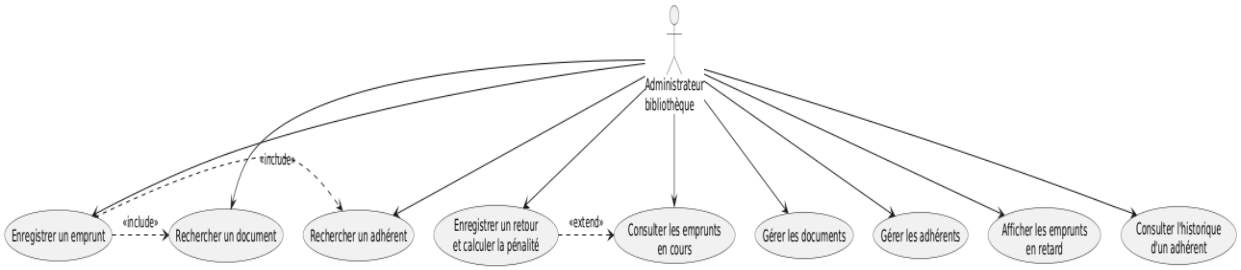
7.1 Diagrammes de classes :



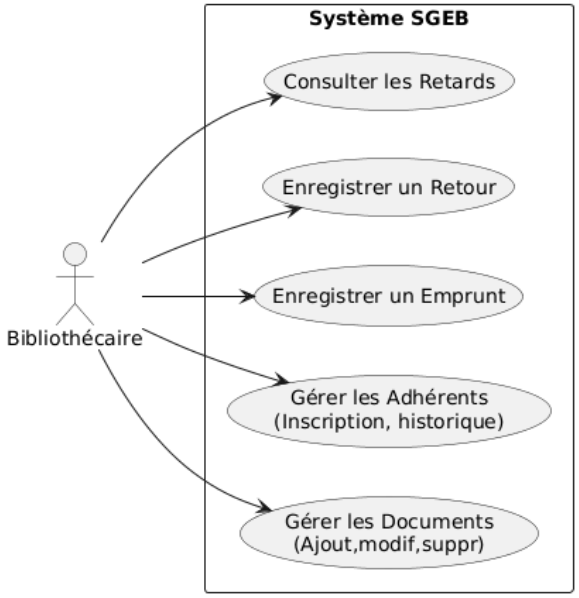
7.2 Diagrammes de la BD :



7.2 Diagramme des cas d'utilisation :



NB : Je suis désolé, je n'ai pas pu faire mieux pour l'image, elle est trop large. Peut-être que la suivante plus simplifiée aidera :



NB : Certains noms de méthodes dans la classe de gestion de la BD sont en anglais, afin d'améliorer la lisibilité et de faciliter la compréhension. Le reste du code est rédigé en français.

8. Conclusion

Ce projet avait pour objectif de concevoir et de développer un système de gestion des emprunts de bibliothèque. L'application réalisée permet de gérer les documents (ajout, modification, suppression, recherche), les adhérents (inscription, mise à jour, suppression, statut) ainsi que les emprunts et retours, avec prise en compte des retards et des pénalités.
Sur le plan technique, le projet s'appuie sur Java pour la logique métier, JavaFX pour l'interface graphique et SQLite pour la persistance des données. La séparation entre la couche métier (BibliothequeManager) et la couche d'accès aux données (DatabaseManager) facilite l'évolution et la maintenance du code.
Ce travail m'a permis de mettre en pratique la programmation orientée objet, la manipulation d'une base de données relationnelle et la création d'une interface utilisateur graphique. Plusieurs améliorations seraient possibles.